Analyze Base Pipeline

Data Ingestion

The pipeline's data source is a manually uploaded CSV file (`AppReviewData.csv`) registered in the Azure ML workspace under the name `user_feedback`. The file contains app review text in a `reviewText` column and a corresponding numeric rating in a `reviewerRating` column, along with additional metadata. The dataset is loaded as a static asset and fed directly into the preprocessing stage. All processing is batch-based, meaning the entire dataset must be present and manually uploaded before any pipeline run can begin.

Limitations / Considerations:

- Data ingestion is not automated which limits scalability and real-time updates.
- There is no schema validation at ingestion. Missing columns, renamed fields, or malformed rows will pass silently into preprocessing, where they may cause failures or corrupt predictions without clear error messages.
- The static CSV format does not include ingestion timestamps, which are required to support time-windowed features such as the 7-day rolling most-complained features display.

Pre-Processing

The preprocessing stage prepares raw review text (`reviewText`) for machine learning by applying a series of structured text-cleaning and normalization steps. The goal is to standardize input text, reduce noise, and improve consistency for downstream modeling.

The pipeline applies the following transformations:

- Expands English verb contractions (e.g., "don't" → "do not")
- Converts all text to lowercase for normalization
- Removes numbers and special characters to reduce noise
- Removes email addresses and URLs to eliminate irrelevant content
- Removes duplicate characters to standardize exaggerated text (e.g., "soooo" → "so")
- Normalizes backslashes to forward slashes for consistency
- Splits tokens based on special characters to improve feature extraction
- Applies stop word removal to eliminate common non-informative words

- Uses lemmatization to reduce words to their base form
- Performs sentence detection for structured text processing
- A custom regular expression and replacement string are also applied to handle dataset-specific text patterns.

Limitations / Considerations:

- Removing special characters and punctuation strips exclamation marks and question marks, which carry meaningful sentiment weight. A review reading "Amazing!!!" is treated identically to "Amazing" after preprocessing, losing a measurable intensity signal that affects prediction accuracy.
- Base pipeline has a weak semantic understanding (does not understand sarcasm, context, or meaning).
- Removing numbers eliminates potentially useful rating-related language (e.g., "5 stars", "rated it 1 out of 5"), which could reinforce or contradict the review text signal.
- The pipeline has no pre-processing gate to catch and flag reviews that should be excluded or reviewed manually before scoring, specifically single-word reviews and empty-text negative reviews, both of which are explicit functional requirements.

Splitting Data

Current Behavior: The split data component is configured as follows:

- Splitting mode: Split Rows
- Fraction of rows in the first output dataset: 0.8
- Randomized split: True
- Random seed: 0
- Stratified split: False

80% of the preprocessed dataset is allocated to the training set and 20% to the test set. The fixed random seed ensures that the same split is reproduced on every pipeline run. Stratification is disabled, meaning no effort is made to ensure that the distribution of rating values is consistent across both splits.

Limitations / Considerations:

- A single fixed train/test split gives only one performance estimate, making it difficult to confidently claim the system meets the 80% accuracy threshold. Cross-validation would give a more reliable measure.
- The reliance on a single randomized split also limits the ability to properly evaluate the reliability of the model.
- With stratification disabled, certain rating values may be underrepresented in one split, reducing evaluation reliability and contributing to model bias.

Model Training

Current Behavior: The pipeline uses a Linear Regression model configured as follows:

- Solution method: Ordinary Least Squares (OLS)
- L2 regularization weight: 0.001
- Include intercept term: True
- Random number seed: not set
- Label column: `reviewerRating`
- Model explanations: False

The model is trained on the 80% training split produced by the Split Data component. It learns to predict a continuous numeric value representing the review rating using the preprocessed text features as input. L2 regularization is applied at a low weight to reduce overfitting. The intercept term allows the model to capture a baseline rating level independent of the text features.

Limitations / Considerations:

- The model is trained on a 0–1 scale when 1–10 is required.
- Linear regression assumes a straight-line relationship between text features and ratings, which does not reflect how sentiment actually works. The same words in different contexts can mean very different things, and a linear model cannot handle that.
- There is no way to generate confidence scores with this model type. Meeting the confidence-related requirements will likely require switching to a classification-based model or adding a custom layer on top of the current output.
- Model explanations are disabled, so there is no way to see which parts of a review

drove a particular prediction, making it harder to spot bias or diagnose errors.

- No experiment tracking is set up, meaning training parameters and results are not logged automatically. This will make it difficult to compare experiments in Phase 3.

Score / Serving

The Score Model component is configured with "Append score columns to output" set to True. After training, the model generates predictions on the 20% test split. The predicted rating values (labeled *Scored Labels*) are appended as an additional column to the original test dataset rows, producing an output table that contains both the original review data and its corresponding prediction side by side.

Limitations / Considerations:

- Predictions are returned on the raw 0–1 scale with no rescaling applied, so every output is on the wrong scale relative to the 1–10 requirement.
- There is no post-scoring logic layer. Flagging rules for confidence thresholds, single-word reviews, spam indicators, and aggregate score thresholds all need to exist between the model output and the end user, and none of them currently do.
- No confidence score is attached to any prediction.
- Scored output is appended to the full row but is not written to any persistent or queryable store, meaning a dashboard or downstream application has no reliable way to consume it.

UI / Dashboard

No UI or dashboard component exists in the current baseline. The only way to view predictions is through the Azure ML Studio pipeline run interface, which requires direct access to the Azure workspace and is not designed for end-user consumption. There is no filtering, sorting, flagging display, trend visualization, or aggregated app score view of any kind.

Limitations / Considerations:

- Trend tracking requires scored results to be stored with timestamps over time. Without a data store retaining at least 6 months of history, before/after app update comparisons and weekly complaint trends cannot be displayed.
- The absence of any UI means no user-facing requirement can be demonstrated or tested at this stage. This is the largest functional gap in the current system.

Iterations

Iteration 1 – Fix the Baseline & Core Pipeline Improvements

This iteration focuses on correcting the most critical gaps in the existing pipeline:

- Rescale model output from 0–1 to 1–10
- Improve preprocessing (preserve sentiment signals, add language detection)
- Implement flagging logic for single-word reviews and empty negative reviews
- Switch to stratified splitting and explore cross-validation
- Explore alternative models that support confidence score generation
- Set up experiment tracking

Iteration 2 – Confidence Scoring, Flagging Logic & Dashboard

This iteration builds on a working pipeline and adds the user-facing layer:

- Implement confidence score generation
- Build the post-scoring business logic layer (spam detection, aggregate score thresholds, confidence-based flagging)
- Build a lightweight Streamlit dashboard covering filtering, sorting, flagging indicators, and trend display
- Set up persistent output storage so the dashboard has data to read from